Does Transfer Learning Help, and Which Training Data Matter?

An Empirical Analysis of a Transfer-Learning Experiment

Melani Evangelou Luca Ionescu Chris Obis Maksym Ziemlewski

1 Data Exploration

The response is a metric score bounded in $[0,1]$, recorded for nine models on seven test datasets (TeD). Transfer models are trained on subsets of eight training datasets (TrD1-TrD8). The nine models are split in three non-transfer baselines (B1-B3), built specifically for the target tasks, and six transfer models: a single-source model (MS) and five multi-source models that refine a different share of the architecture for the target task, from none (MN) through one to three parts (M1-M3), and to the full 4-part model (MF). The most striking feature of the data is that performance is **dominated by the test set** (Fig. 1). Mean scores range from 0.05 on the TeD5 **recommendation task** to 0.79 on the TeD1 **classification task**, and the within-test-set standard deviation ranges from 0.008 on TeD2 to 0.111 on TeD1, a $\sim 15\times$ spread. There is also a visible task-type clustering pattern, with the four classification sets sitting at the higher end, the regression sets lower, and recommendation lowest. These heterogeneous results have two consequences that shape the whole analysis. First, the test set is a dominant nuisance factor that must be controlled for. Second, raw score differences are not comparable across test sets. For example, a 0.02 gain on TeD5, whose scores live in a 0.06-wide band, is far more relevant than the same 0.02 on TeD1. We therefore choose to standardize the score within each test set before estimating effects.

2 Modelling Approach

A large preliminary linear model with all factors and their interactions included, ranked by Type-II ANOVA (Table 1), confirms the exploration: the test set explains 92.6% of the variance, overshadowing every other effect. A high overall coefficient of determination R^2 is therefore uninformative since it merely reflects task differences. Thus, all estimates below are computed after accounting for the test set. For this, we model the **within-TeD standardized score**, where each test set is centred and scaled to unit variance. An effect is then expressed as "**standard deviations of that test set**" or "**SD units**". This transformation also gives near-perfect variance homogeneity, as the pronounced funnel in the raw residuals vanishes and the residuals become approximately Normal (Fig. 2).

We then exploit the experiment's structure with a purpose-built model per question. For RQ1 we contrast each architecture against the baselines within each test set. For RQ2 we add three factors implicit in the design: the number of training datasets, the refinement depth, and the eight dataset indicators, each interacted with the test set. These cannot all be put in one model because they are confounded, as the number of datasets is tied to architecture: baselines use 0, MS exactly 1, the multi-source models 2–8, and the dataset indicators vary only among transfer runs. We therefore fit each effect on the subset where it actually varies. Table 2 gives the specification of every model. Effects are summarised as estimated marginal means with 95% confidence intervals. To check that no distortion was introduced by the transform to arrive at the conclusions, every result was reproduced with a logit model and a quasi-binomial GLM that respect the $[0,1]$ bound. All three approaches agree on the direction and significance of each effect.

Table 1: Preliminary model, Type-II ANOVA (raw score response). The test set dominates. The effects of interest are tiny shares of the total variance, but still statistically significant, and are hereby analyzed after controlling for (or standardizing within) the test set.

Term	df	% of total SS	F
Test set (TeD)	6	92.6	8963
Architecture (model)	6	1.3	125
TeD × architecture	36	0.6	9.8
TeD × #datasets	7	0.4	36.3
Residual	2940	5.1	—

Table 2: Model specification for each analysis (Wilkinson notation: "$A \times B$" = main effects and interaction). The response is the within-TeD standardized score z (effects in SD units). The same specifications were refit with logit(score) and a quasi-binomial GLM as robustness checks.
arch.=architecture (B,MS,MN,M1–MF), n=number of training datasets, *multi*=MN,M1–MF.

Analysis	Model	Data
Preliminary (Tab. 1)	score $\sim$ TeD $\times$ arch. $+$ TeD:n	all
RQ1 transfer vs. baseline	$z \sim$ TeD $\times$ arch.	all
RQ1 refinement depth	$z \sim$ TeD $+$ depth	multi
RQ2 number of datasets	$z \sim$ TeD $+ n\,(+n^2)$	multi
RQ2 dataset identity	$z \sim$ TeD $\times$ (TrD1$+\cdots+$TrD8)	$n \geq 1$

3 RQ1: Does Transfer Learning Help?

Yes, on almost every task, and the benefit is large if contextualized. On the raw scale, the transfer-over-baseline gap looks negligible for several test sets (Fig. 3a): e.g. $+0.04$ on TeD2 and $+0.02$ on TeD5. However, those test sets have tiny spreads, so in context the gains are substantial. When measured in SD units (Fig. 3b), transfer learning significantly outperforms the baselines on **six of the seven** test sets, by be-

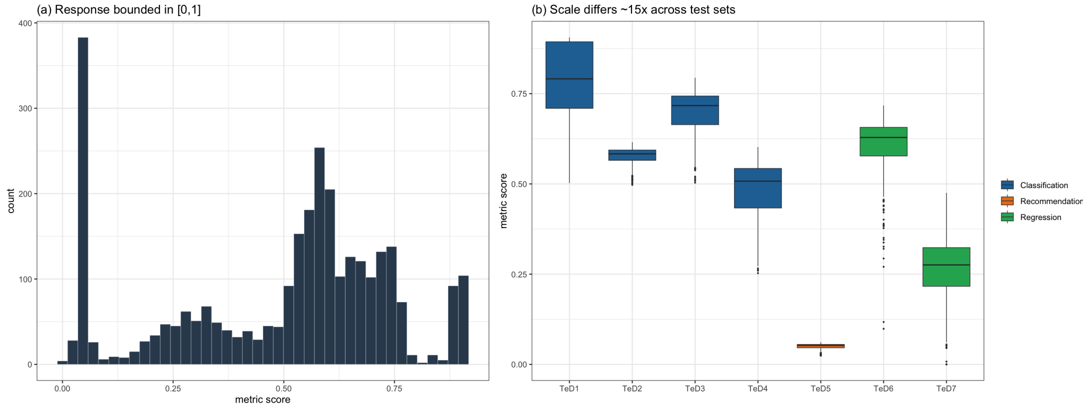

Figure 1: The score is bounded in $[0,1]$ (a) and its scale differs by $\sim 15\times$ across test sets, grouped by task type (b). The test set must be controlled for, and effects must be made comparable across test sets.

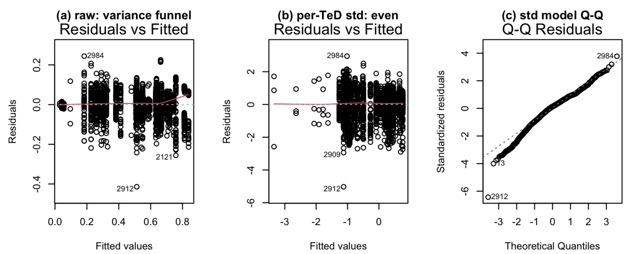

Figure 2: Goodness of fit. The variance funnel of the raw-score model (a) disappears after per-test-set standardization (b). The standardized residuals are approximately Normal apart from a short left tail from a few near-zero TeD7 scores (c).

tween 1.6 and 3.3 SD. The exception is TeD4, where the effect is indistinguishable from zero (0.0 ± 0.9 SD). Averaged over test sets the advantage is 1.9 SD (95% CI $[1.5, 2.2]$). In absolute terms, the largest wins are on the lowest-scoring tasks, where transfer roughly doubles the score on the regression sets TeD7 ($0.10 \rightarrow 0.27$) and TeD6 ($0.33 \rightarrow 0.61$). A different analysis reveals that the advantage grows with the share of the model that is refined for the target task (Fig. 4). Contrasting each architecture against the baseline control gives a clean ordering: the single-source model MS ($+0.9$ SD) and the unrefined multi-source model MN ($+1.2$ SD) help least, while the refined multi-source models improve monotonically from M1 ($+2.1$ SD) to the fully refined MF ($+2.5$ SD). All are significant at $p < 10^{-6}$. Equivalently, each additional refined part adds 0.28 SD (95% CI $[0.26, 0.31]$). Overall, transfer learning should be preferred over task-specific baseline in almost all cases, refining as much of the architecture as the budget allows. The exceptions are cases like TeD4 where no real benefit is identified, so gains should be validated per task.

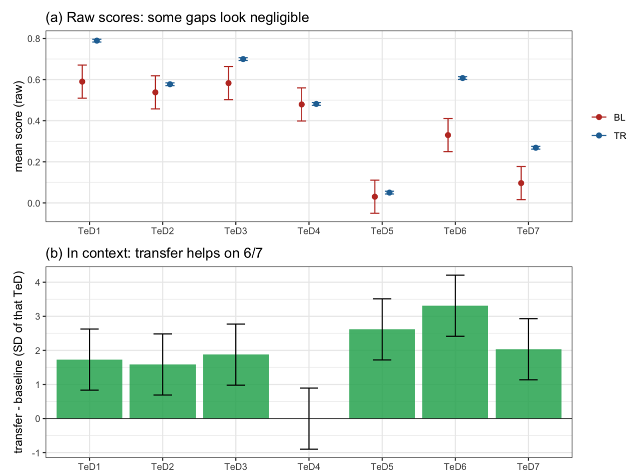

Figure 3: RQ1. (a) On the raw scale, transfer-baseline gaps look small on several test sets. (b) Read in context (SD of each test set), transfer helps on six of seven test sets; only TeD4 is null. Bars are 95% confidence intervals.

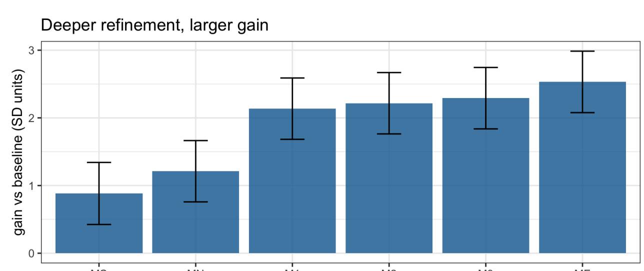

Figure 4: RQ1. Each transfer architecture's gain over the baseline (SD units, 95% CIs). The single-source model MS gains least. Within the multi-source family the gain grows with refinement depth (MN<M1<M2<M3<MF).

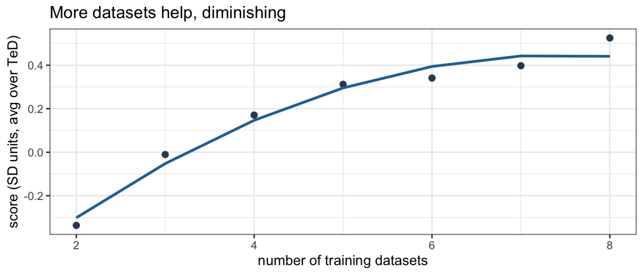

Figure 5: RQ2. Score rises with the number of training datasets, with diminishing returns (SD units, averaged over test sets, 95% CIs).

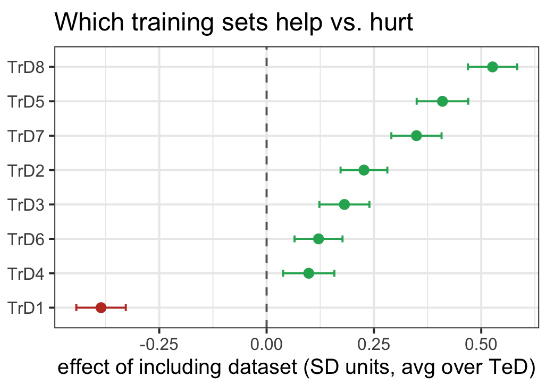

Figure 6: RQ2. Effect of including each training dataset (SD units, averaged over test sets, 95% CIs). TrD1 consistently reduces performance.

4 RQ2: Which Training Data Matter?

Both how many datasets and which ones, but identity matters more.

Adding training datasets helps, as within the multi-source models each extra dataset raises the score by 0.14 SD (95% CI $[0.12, 0.16]$), resulting in a total of ~0.8 SD from two to eight datasets. The gain shows diminishing returns: a significant downward curvature flattens the curve beyond about five datasets (Fig. 5). The choice of *which* datasets to include is a stronger factor (Fig. 6). Including TrD8, TrD5 or TrD7 raises the score most ($+0.5$, $+0.4$, $+0.3$ SD), while TrD2, TrD3, TrD6 and TrD4 help modestly, and **TrD1 consistently *hurts*** (-0.4 SD). Moreover, a dataset's usefulness strongly depends on the target task, so no single dataset is universally best. A model that includes the interaction between the training set and test set reduces *RSS* by 20% ($2029 \rightarrow 1616$, $p \approx 10^{-110}$). For instance, TrD2 is the most valuable source for TeD1 ($+1.7$ SD) yet is useless or harmful on most other tasks, whereas TrD8 helps consistently across nearly all test sets, and TrD1 lowers performance on *every* test set.

Choosing the appropriate source datasets to transfer from increases gain more than simply adding more of them. Useful datasets should be identified per target task, clearly unhelpful ones (such as TrD1) excluded, and only a couple of them combined, since returns diminish quickly.

5 Validity and Limitations

Several factors could have impacted our claims. Design-wise, the baselines were run once per test set ($n=3$ per TeD), so baseline means, and hence the RQ1 gaps, carry wide uncertainty (Fig. 3a). Also, because the number of datasets is confounded with architecture (baselines use 0, MS exactly 1, Mx 2–8), the "number" and "architecture" effects can only be separated within the multi-source family. In terms of measurements, the scores are compared across tasks of different nature, and, even with our within-test-set standardization's effect, their substantive meaning is different per task. Another limitation is that no metadata is available for the training data, which makes it impossible to explain why TrD1 hurts, TrD8 helps, or if there was any information leakage between training and test data. Moreover, because the sample is large, almost any non-zero effect is "significant", therefore we emphasize effect sizes and intervals, and read significance as secondary.

Finally, all findings come from a single experiment and seven tasks, meaning replication is needed to prove generalization.

6 AI Usage

AI methods were used to improve writing coherence and during brainstorming sessions of the team. For programming, the team adopted it to generate visually-appealing plots in R.